

WEEK 5 ASSIGNMENT

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer:

Feature	Traditional MapReduce	Apache Spark
Intermediate Data Storage	Writes intermediate results to disk after each stage	Stores intermediate data in memory (RAM)
Processing Speed	Slower due to frequent disk I/O	Faster because of in-memory computing
Iterative Processing	Not efficient for iterative algorithms	Highly efficient for iterative workloads
Machine Learning Support	Requires repeated disk reads	Reuses cached data in memory
Programming Complexity	Complex Map and Reduce programming model	Simple APIs in Python, Scala, Java, and R
Real-Time Processing	Limited support	Supports near real-time processing
Built-in Libraries	Limited ecosystem	Provides SQL, MLlib, Streaming, and GraphX

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

Spark uses **In-Memory Computing**, where intermediate data is stored in RAM instead of being written to disk after every operation. Machine learning algorithms often process the same dataset multiple times. Spark reuses the data directly from memory, reducing disk I/O and significantly improving execution speed compared to disk-based systems like MapReduce.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

The dropDuplicates() method is used to remove duplicate records based on specified columns. In this case, duplicate rows are identified using the user_id and transaction_date columns.

Code:

```
df_clean = df.dropDuplicates(["user_id", "transaction_date"])
df_clean.show()
```

Output:

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username	status	raw_timestamp
1001	01-01-2025	West	Electronics	100	Delhi	18	Premium	NULL	S101	NULL	NULL	NULL	01-01-2025 10:00
1002	02-01-2025	West	Furniture	105	Delhi	19	Basic	105	S101	user1002@gmail.com	user1002	Completed	02-01-2025 10:01
1003	03-01-2025	West	Clothing	110	Delhi	20	Premium	110	S101	user1003@gmail.com	user1003	Completed	03-01-2025 10:02
1004	04-01-2025	West	Grocery	115	Delhi	21	Basic	115	S101	user1004@gmail.com	user1004	Completed	04-01-2025 10:03
1005	05-01-2025	West	Electronics	120	Delhi	22	Premium	120	S101	user1005@gmail.com	user1005	Completed	05-01-2025 10:04
1006	06-01-2025	West	Furniture	125	Delhi	23	Basic	125	S101	user1006@gmail.com	user1006	Completed	06-01-2025 10:05
1007	07-01-2025	West	Clothing	130	Delhi	24	Premium	130	S101	user1007@gmail.com	user1007	Completed	07-01-2025 10:06
1008	08-01-2025	West	Grocery	135	Delhi	25	Basic	135	S101	user1008@gmail.com	user1008	Completed	08-01-2025 10:07
1009	09-01-2025	West	Electronics	140	Delhi	26	Premium	140	S101	user1009@gmail.com	user1009	Completed	09-01-2025 10:08
1010	10-01-2025	West	Furniture	145	Delhi	27	Basic	145	S101	user1010@gmail.com	user1010	Completed	10-01-2025 10:09
1011	11-01-2025	West	Clothing	150	Delhi	28	Premium	150	S101	user1011@gmail.com	user1011	Completed	11-01-2025 10:10
1012	12-01-2025	West	Grocery	155	Delhi	29	Basic	155	S101	user1012@gmail.com	user1012	Completed	12-01-2025 10:11
1013	13-01-2025	West	Electronics	160	Delhi	30	Premium	160	S101	user1013@gmail.com	user1013	Completed	13-01-2025 10:12
1014	14-01-2025	West	Furniture	165	Delhi	31	Basic	165	S101	user1014@gmail.com	user1014	Completed	14-01-2025 10:13
1015	15-01-2025	West	Clothing	170	Delhi	32	Premium	170	S101	user1015@gmail.com	user1015	Completed	15-01-2025 10:14
1016	16-01-2025	West	Grocery	175	Delhi	33	Basic	175	S101	user1016@gmail.com	user1016	Completed	16-01-2025 10:15
1017	17-01-2025	West	Electronics	180	Delhi	34	Premium	180	S101	user1017@gmail.com	user1017	Completed	17-01-2025 10:16
1018	18-01-2025	West	Furniture	185	Delhi	35	Basic	NULL	S101	user1018@gmail.com	user1018	Completed	18-01-2025 10:17
1019	19-01-2025	West	Clothing	190	Delhi	18	Premium	190	S101	user1019@gmail.com	user1019	Completed	19-01-2025 10:18
1020	20-01-2025	West	Grocery	195	Delhi	19	Basic	195	S101	NULL	user1020	Completed	20-01-2025 10:19

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Code:

```
from pyspark.sql.functions import col, avg
df.filter(col("region") == "West") \
    .groupBy("product_category") \
```

```
.agg(avg("sale_amount").alias("Average_Sale")) \
.show()
```

Output:

```
+-----+-----+
|product_category|Average_Sale|
+-----+-----+
|      Grocery   |      405.0 |
| Electronics    |      390.0 |
|      Clothing  |      400.0 |
|      Furniture |      395.0 |
+-----+-----+
```

Q5. What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer

Method	Description
<code>.na.drop()</code>	Removes rows that contain null values in one or more columns.
<code>.na.fill()</code>	Replaces null values with a specified value instead of removing the rows.

Example of `.na.drop()`

```
df.na.drop().show()
```

This removes all rows containing null values.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount| city|age|subscription|price|store_id|email|username| status| raw_timestamp|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1002|    02-01-2025| West|    Furniture|      105|Delhi| 19|    Basic|    105|    S101|user1002@gmail.com|user1002|Completed|02-01-2025 10:01|
| 1003|    03-01-2025| West|    Clothing|      110|Delhi| 20|   Premium|    110|    S101|user1003@gmail.com|user1003|Completed|03-01-2025 10:02|
| 1004|    04-01-2025| West|    Grocery|      115|Delhi| 21|    Basic|    115|    S101|user1004@gmail.com|user1004|Completed|04-01-2025 10:03|
| 1005|    05-01-2025| West| Electronics|      120|Delhi| 22|   Premium|    120|    S101|user1005@gmail.com|user1005|Completed|05-01-2025 10:04|
| 1006|    06-01-2025| West|    Furniture|      125|Delhi| 23|    Basic|    125|    S101|user1006@gmail.com|user1006|Completed|06-01-2025 10:05|
| 1007|    07-01-2025| West|    Clothing|      130|Delhi| 24|   Premium|    130|    S101|user1007@gmail.com|user1007|Completed|07-01-2025 10:06|
| 1008|    08-01-2025| West|    Grocery|      135|Delhi| 25|    Basic|    135|    S101|user1008@gmail.com|user1008|Completed|08-01-2025 10:07|
| 1009|    09-01-2025| West| Electronics|      140|Delhi| 26|   Premium|    140|    S101|user1009@gmail.com|user1009|Completed|09-01-2025 10:08|
| 1010|   10-01-2025| West|    Furniture|      145|Delhi| 27|    Basic|    145|    S101|user1010@gmail.com|user1010|Completed|10-01-2025 10:09|
| 1011|   11-01-2025| West|    Clothing|      150|Delhi| 28|   Premium|    150|    S101|user1011@gmail.com|user1011|Completed|11-01-2025 10:10|
| 1012|   12-01-2025| West|    Grocery|      155|Delhi| 29|    Basic|    155|    S101|user1012@gmail.com|user1012|Completed|12-01-2025 10:11|
| 1013|   13-01-2025| West| Electronics|      160|Delhi| 30|   Premium|    160|    S101|user1013@gmail.com|user1013|Completed|13-01-2025 10:12|
| 1014|   14-01-2025| West|    Furniture|      165|Delhi| 31|    Basic|    165|    S101|user1014@gmail.com|user1014|Completed|14-01-2025 10:13|
| 1015|   15-01-2025| West|    Clothing|      170|Delhi| 32|   Premium|    170|    S101|user1015@gmail.com|user1015|Completed|15-01-2025 10:14|
| 1016|   16-01-2025| West|    Grocery|      175|Delhi| 33|    Basic|    175|    S101|user1016@gmail.com|user1016|Completed|16-01-2025 10:15|
| 1017|   17-01-2025| West| Electronics|      180|Delhi| 34|   Premium|    180|    S101|user1017@gmail.com|user1017|Completed|17-01-2025 10:16|
| 1019|   19-01-2025| West|    Clothing|      190|Delhi| 18|   Premium|    190|    S101|user1019@gmail.com|user1019|Completed|19-01-2025 10:18|
| 1021|   21-01-2025| West| Electronics|      200|Delhi| 20|   Premium|    200|    S101|user1021@gmail.com|user1021|Completed|21-01-2025 10:20|
| 1022|   22-01-2025| West|    Furniture|      205|Delhi| 21|    Basic|    205|    S101|user1022@gmail.com|user1022|Completed|22-01-2025 10:21|
| 1023|   23-01-2025| West|    Clothing|      210|Delhi| 22|   Premium|    210|    S101|user1023@gmail.com|user1023|Completed|23-01-2025 10:22|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Example of `.na.fill()`

```
df_filled = df.na.fill({"status": "Unknown"})
df_filled.show()
```

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username	status	raw_timestamp
1001	01-01-2025	West	Electronics	100	Delhi	18	Premium	NULL	S101	NULL	NULL	Unknown	01-01-2025 10:00
1002	02-01-2025	West	Furniture	105	Delhi	19	Basic	105	S101 user1002@gmail.com		user1002	Completed	02-01-2025 10:01
1003	03-01-2025	West	Clothing	110	Delhi	20	Premium	110	S101 user1003@gmail.com		user1003	Completed	03-01-2025 10:02
1004	04-01-2025	West	Grocery	115	Delhi	21	Basic	115	S101 user1004@gmail.com		user1004	Completed	04-01-2025 10:03
1005	05-01-2025	West	Electronics	120	Delhi	22	Premium	120	S101 user1005@gmail.com		user1005	Completed	05-01-2025 10:04
1006	06-01-2025	West	Furniture	125	Delhi	23	Basic	125	S101 user1006@gmail.com		user1006	Completed	06-01-2025 10:05
1007	07-01-2025	West	Clothing	130	Delhi	24	Premium	130	S101 user1007@gmail.com		user1007	Completed	07-01-2025 10:06
1008	08-01-2025	West	Grocery	135	Delhi	25	Basic	135	S101 user1008@gmail.com		user1008	Completed	08-01-2025 10:07
1009	09-01-2025	West	Electronics	140	Delhi	26	Premium	140	S101 user1009@gmail.com		user1009	Completed	09-01-2025 10:08
1010	10-01-2025	West	Furniture	145	Delhi	27	Basic	145	S101 user1010@gmail.com		user1010	Completed	10-01-2025 10:09
1011	11-01-2025	West	Clothing	150	Delhi	28	Premium	150	S101 user1011@gmail.com		user1011	Completed	11-01-2025 10:10
1012	12-01-2025	West	Grocery	155	Delhi	29	Basic	155	S101 user1012@gmail.com		user1012	Completed	12-01-2025 10:11
1013	13-01-2025	West	Electronics	160	Delhi	30	Premium	160	S101 user1013@gmail.com		user1013	Completed	13-01-2025 10:12
1014	14-01-2025	West	Furniture	165	Delhi	31	Basic	165	S101 user1014@gmail.com		user1014	Completed	14-01-2025 10:13
1015	15-01-2025	West	Clothing	170	Delhi	32	Premium	170	S101 user1015@gmail.com		user1015	Completed	15-01-2025 10:14
1016	16-01-2025	West	Grocery	175	Delhi	33	Basic	175	S101 user1016@gmail.com		user1016	Completed	16-01-2025 10:15
1017	17-01-2025	West	Electronics	180	Delhi	34	Premium	180	S101 user1017@gmail.com		user1017	Completed	17-01-2025 10:16
1018	18-01-2025	West	Furniture	185	Delhi	35	Basic	NULL	S101 user1018@gmail.com		user1018	Completed	18-01-2025 10:17
1019	19-01-2025	West	Clothing	190	Delhi	18	Premium	190	S101 user1019@gmail.com		user1019	Completed	19-01-2025 10:18
1020	20-01-2025	West	Grocery	195	Delhi	19	Basic	195	S101	NULL	user1020	Completed	20-01-2025 10:19

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Code

```
from pyspark.sql.functions import count

df.groupBy("city") \

    .agg(count("*").alias("record_count")) \

    .filter("record_count > 100") \

    .show()
```

Output:

city	record_count
Mumbai	110
Kolkata	110
Delhi	120

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer

Spark DataFrames are **immutable**, which means that once a DataFrame is created, its data cannot be changed directly. Any transformation such as dropping a column, renaming a column, or filtering rows creates a **new DataFrame** instead of modifying the existing one.

This ensures data consistency, fault tolerance, and enables Spark to optimize execution through its lazy evaluation mechanism.

Example

Dropping a Column

```
df_new = df.drop("status")
```

Renaming a Column

```
df_renamed = df.withColumnRenamed(
    "sale_amount",
    "sales_amount"
)
```

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Code

```
from pyspark.sql.functions import col

df.filter(
    (col("age").between(18, 30)) &
    (col("subscription") == "Premium")
).show()
```

Output:

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username	status	raw_timestamp
1001	01-01-2025	West	Electronics	100	Delhi	18	Premium	NULL	S101	NULL	NULL	NULL	01-01-2025 10:00
1003	03-01-2025	West	Clothing	110	Delhi	20	Premium	110	S101	user1003@gmail.com	user1003	Completed	03-01-2025 10:02
1005	05-01-2025	West	Electronics	120	Delhi	22	Premium	120	S101	user1005@gmail.com	user1005	Completed	05-01-2025 10:04
1007	07-01-2025	West	Clothing	130	Delhi	24	Premium	130	S101	user1007@gmail.com	user1007	Completed	07-01-2025 10:06
1009	09-01-2025	West	Electronics	140	Delhi	26	Premium	140	S101	user1009@gmail.com	user1009	Completed	09-01-2025 10:08
1011	11-01-2025	West	Clothing	150	Delhi	28	Premium	150	S101	user1011@gmail.com	user1011	Completed	11-01-2025 10:10
1013	13-01-2025	West	Electronics	160	Delhi	30	Premium	160	S101	user1013@gmail.com	user1013	Completed	13-01-2025 10:12
1019	19-01-2025	West	Clothing	190	Delhi	18	Premium	190	S101	user1019@gmail.com	user1019	Completed	19-01-2025 10:18
1021	21-01-2025	West	Electronics	200	Delhi	20	Premium	200	S101	user1021@gmail.com	user1021	Completed	21-01-2025 10:20
1023	23-01-2025	West	Clothing	210	Delhi	22	Premium	210	S101	user1023@gmail.com	user1023	Completed	23-01-2025 10:22
1025	25-01-2025	West	Electronics	220	Delhi	24	Premium	220	S101	user1025@gmail.com	user1025	Completed	25-01-2025 10:24
1027	27-01-2025	West	Clothing	230	Delhi	26	Premium	230	S101	user1027@gmail.com	user1027	Completed	27-01-2025 10:26
1029	01-01-2025	West	Electronics	240	Delhi	28	Premium	240	S101	user1029@gmail.com	user1029	Completed	01-01-2025 10:28
1031	03-01-2025	West	Clothing	250	Delhi	30	Premium	250	S101	user1031@gmail.com	user1031	Completed	03-01-2025 10:30
1037	09-01-2025	West	Electronics	280	Delhi	18	Premium	280	S101	user1037@gmail.com	user1037	Completed	09-01-2025 10:36
1039	11-01-2025	West	Clothing	290	Delhi	20	Premium	290	S101	NULL	user1039	Completed	11-01-2025 10:38
1041	13-01-2025	West	Electronics	300	Delhi	22	Premium	300	S101	user1041@gmail.com	user1041	Completed	13-01-2025 10:40
1043	15-01-2025	West	Clothing	310	Delhi	24	Premium	310	S101	user1043@gmail.com	user1043	Completed	15-01-2025 10:42
1045	17-01-2025	West	Electronics	320	Delhi	26	Premium	320	S101	user1045@gmail.com	user1045	Completed	17-01-2025 10:44
1047	19-01-2025	West	Clothing	330	Delhi	28	Premium	330	S101	user1047@gmail.com	NULL	Completed	19-01-2025 10:46

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer

Handling null values before performing mathematical aggregations is important because null values can lead to inaccurate or misleading results. Functions such as sum(), avg(), min(), and max() may ignore null values, which can affect calculations and produce incorrect insights.

By cleaning or replacing null values beforehand, you ensure that aggregations are performed on complete and consistent data.

Reasons

Reason	Explanation
Improves Accuracy	Prevents incorrect calculations caused by missing values.
Ensures Consistency	Aggregations are performed on standardized data.
Avoids Data Loss	Filling nulls preserves records that might otherwise be excluded.
Produces Reliable Insights	Results better reflect the actual dataset.

Example

```
df_clean = df.na.fill({"price": 0})

from pyspark.sql.functions import avg

df_clean.agg(avg("price")).show()
```

Output:

```
+-----+
|avg(price)|
+-----+
|   342.575|
+-----+
```

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

Code

```
from pyspark.sql.functions import col, to_timestamp

df_updated = (

    df.withColumn(

        "event_time",

        to_timestamp(col("raw_timestamp"), "dd-MM-yyyy HH:mm")

    )

    .drop("raw_timestamp")

)

df_updated.show()
```

Output:

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username	status	event_time
1001	01-01-2025	West	Electronics	100	Delhi	18	Premium	NULL	S101	NULL	NULL	NULL	2025-01-01 10:00:00
1002	02-01-2025	West	Furniture	105	Delhi	19	Basic	105	S101	user1002@gmail.com	user1002	Completed	2025-01-02 10:01:00
1003	03-01-2025	West	Clothing	110	Delhi	20	Premium	110	S101	user1003@gmail.com	user1003	Completed	2025-01-03 10:02:00
1004	04-01-2025	West	Grocery	115	Delhi	21	Basic	115	S101	user1004@gmail.com	user1004	Completed	2025-01-04 10:03:00
1005	05-01-2025	West	Electronics	120	Delhi	22	Premium	120	S101	user1005@gmail.com	user1005	Completed	2025-01-05 10:04:00
1006	06-01-2025	West	Furniture	125	Delhi	23	Basic	125	S101	user1006@gmail.com	user1006	Completed	2025-01-06 10:05:00
1007	07-01-2025	West	Clothing	130	Delhi	24	Premium	130	S101	user1007@gmail.com	user1007	Completed	2025-01-07 10:06:00
1008	08-01-2025	West	Grocery	135	Delhi	25	Basic	135	S101	user1008@gmail.com	user1008	Completed	2025-01-08 10:07:00
1009	09-01-2025	West	Electronics	140	Delhi	26	Premium	140	S101	user1009@gmail.com	user1009	Completed	2025-01-09 10:08:00
1010	10-01-2025	West	Furniture	145	Delhi	27	Basic	145	S101	user1010@gmail.com	user1010	Completed	2025-01-10 10:09:00
1011	11-01-2025	West	Clothing	150	Delhi	28	Premium	150	S101	user1011@gmail.com	user1011	Completed	2025-01-11 10:10:00
1012	12-01-2025	West	Grocery	155	Delhi	29	Basic	155	S101	user1012@gmail.com	user1012	Completed	2025-01-12 10:11:00
1013	13-01-2025	West	Electronics	160	Delhi	30	Premium	160	S101	user1013@gmail.com	user1013	Completed	2025-01-13 10:12:00
1014	14-01-2025	West	Furniture	165	Delhi	31	Basic	165	S101	user1014@gmail.com	user1014	Completed	2025-01-14 10:13:00
1015	15-01-2025	West	Clothing	170	Delhi	32	Premium	170	S101	user1015@gmail.com	user1015	Completed	2025-01-15 10:14:00
1016	16-01-2025	West	Grocery	175	Delhi	33	Basic	175	S101	user1016@gmail.com	user1016	Completed	2025-01-16 10:15:00
1017	17-01-2025	West	Electronics	180	Delhi	34	Premium	180	S101	user1017@gmail.com	user1017	Completed	2025-01-17 10:16:00
1018	18-01-2025	West	Furniture	185	Delhi	35	Basic	NULL	S101	user1018@gmail.com	user1018	Completed	2025-01-18 10:17:00
1019	19-01-2025	West	Clothing	190	Delhi	18	Premium	190	S101	user1019@gmail.com	user1019	Completed	2025-01-19 10:18:00
1020	20-01-2025	West	Grocery	195	Delhi	19	Basic	195	S101	NULL	user1020	Completed	2025-01-20 10:19:00

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer

A **Shuffle** is the process in Spark where data is redistributed across partitions so that records with the same key are brought together for operations such as `groupBy()`, `join()`, `distinct()`, and `reduceByKey()`.

During a grouping operation, Spark moves data between partitions over the network to ensure that all records belonging to the same group are placed in the same partition before aggregation is performed.

Example

```
df.groupBy("store_id").sum("price")  
... DataFrame[store_id: string, sum(price): bigint]  
  
from pyspark.sql.functions import sum  
  
df.groupBy("store_id") \  
  .agg(sum("price").alias("total_revenue")) \  
  .show()  
...  
+-----+-----+  
|store_id|total_revenue|  
+-----+-----+  
|    S102|         39985|  
|    S104|         12540|  
|    S101|         44520|  
|    S103|         39985|  
+-----+-----+
```

When this query runs:

1. Spark identifies records with the same store_id.
2. Data is transferred across partitions.
3. Records with the same key are collected together.
4. Aggregation (sum) is performed on each group.

Why is Shuffle a Wide Transformation?

A transformation is called **wide** when data from multiple partitions is required to produce the output.

Narrow Transformation	Wide Transformation
Data remains within the same partition	Data moves across partitions
Faster execution	More expensive operation
Examples: filter(), map()	Examples: groupBy(), join(), distinct()

Since `groupBy()` requires data exchange between partitions, it triggers a shuffle and is therefore classified as a **wide transformation**.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Code

```
from pyspark.sql.functions import col
```

```
df_clean = df.filter(
    col("email").isNotNull() &
    (col("username") != "")
)
```

```
df_clean.show()
```

Explanation

- `col("email").isNotNull()` keeps only rows where the email is not null.
- `(col("username") != "")` keeps only rows where the username is not empty.
- The `&` operator combines both conditions using a logical AND.
- Rows with a null email or an empty username are removed from the DataFrame.

Output:

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	price	store_id	email	username	status	raw_timestamp
1002	02-01-2025	West	Furniture	105	Delhi	19	Basic	105	S101	user1002@gmail.com	user1002	Completed	02-01-2025 10:01
1003	03-01-2025	West	Clothing	110	Delhi	20	Premium	110	S101	user1003@gmail.com	user1003	Completed	03-01-2025 10:02
1004	04-01-2025	West	Grocery	115	Delhi	21	Basic	115	S101	user1004@gmail.com	user1004	Completed	04-01-2025 10:03
1005	05-01-2025	West	Electronics	120	Delhi	22	Premium	120	S101	user1005@gmail.com	user1005	Completed	05-01-2025 10:04
1006	06-01-2025	West	Furniture	125	Delhi	23	Basic	125	S101	user1006@gmail.com	user1006	Completed	06-01-2025 10:05
1007	07-01-2025	West	Clothing	130	Delhi	24	Premium	130	S101	user1007@gmail.com	user1007	Completed	07-01-2025 10:06
1008	08-01-2025	West	Grocery	135	Delhi	25	Basic	135	S101	user1008@gmail.com	user1008	Completed	08-01-2025 10:07
1009	09-01-2025	West	Electronics	140	Delhi	26	Premium	140	S101	user1009@gmail.com	user1009	Completed	09-01-2025 10:08
1010	10-01-2025	West	Furniture	145	Delhi	27	Basic	145	S101	user1010@gmail.com	user1010	Completed	10-01-2025 10:09
1011	11-01-2025	West	Clothing	150	Delhi	28	Premium	150	S101	user1011@gmail.com	user1011	Completed	11-01-2025 10:10
1012	12-01-2025	West	Grocery	155	Delhi	29	Basic	155	S101	user1012@gmail.com	user1012	Completed	12-01-2025 10:11
1013	13-01-2025	West	Electronics	160	Delhi	30	Premium	160	S101	user1013@gmail.com	user1013	Completed	13-01-2025 10:12
1014	14-01-2025	West	Furniture	165	Delhi	31	Basic	165	S101	user1014@gmail.com	user1014	Completed	14-01-2025 10:13
1015	15-01-2025	West	Clothing	170	Delhi	32	Premium	170	S101	user1015@gmail.com	user1015	Completed	15-01-2025 10:14
1016	16-01-2025	West	Grocery	175	Delhi	33	Basic	175	S101	user1016@gmail.com	user1016	Completed	16-01-2025 10:15
1017	17-01-2025	West	Electronics	180	Delhi	34	Premium	180	S101	user1017@gmail.com	user1017	Completed	17-01-2025 10:16
1018	18-01-2025	West	Furniture	185	Delhi	35	Basic	NULL	S101	user1018@gmail.com	user1018	Completed	18-01-2025 10:17
1019	19-01-2025	West	Clothing	190	Delhi	18	Premium	190	S101	user1019@gmail.com	user1019	Completed	19-01-2025 10:18
1021	21-01-2025	West	Electronics	200	Delhi	20	Premium	200	S101	user1021@gmail.com	user1021	Completed	21-01-2025 10:20
1022	22-01-2025	West	Furniture	205	Delhi	21	Basic	205	S101	user1022@gmail.com	user1022	Completed	22-01-2025 10:21

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer

The `.agg()` function allows multiple aggregate calculations to be performed in a single operation. It is commonly used to compute statistics such as minimum, maximum, average, sum, and count.

Code

```
from pyspark.sql.functions import min, max, avg
df.agg(
    min("price").alias("Minimum_Price"),
    max("price").alias("Maximum_Price"),
    avg("price").alias("Average_Price")
).show()
```

Output:

```
+-----+-----+-----+
|Minimum_Price|Maximum_Price|Average_Price|
+-----+-----+-----+
|          105|          690|349.56632653061223|
+-----+-----+-----+
```

Q14: In the context of cleaning a dataset, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Answer

Using `inferSchema=true` allows Spark to automatically determine the data type of each column. However, when a dataset contains messy or inconsistent date formats, Spark may incorrectly infer the column data type or fail to parse some values.

This can result in null values, incorrect data types, or errors during data processing and analysis.

Risks

Risk	Description
Incorrect Data Type	Spark may infer <code>StringType</code> instead of <code>DateType</code> or <code>TimestampType</code> .
Null Values	Invalid date formats may be converted to null.
Data Quality Issues	Inconsistent parsing can lead to inaccurate results.

Processing Errors	Date-based operations may fail or produce incorrect outputs.
-------------------	--

Example Code

```
df = spark.read.csv(
    "smart_retail_operations.csv",
    header=True,
    inferSchema=True
)
```

If date formats are inconsistent, Spark may not correctly interpret the date column.

Best Practice

Instead of relying on `inferSchema=true`, define the schema explicitly or use functions such as `to_date()` or `to_timestamp()` with the correct format.

```
from pyspark.sql.functions import to_timestamp
df = df.withColumn(
    "event_time",
    to_timestamp("raw_timestamp", "dd-MM-yyyy HH:mm")
)
```

Q15: Write a final processing pipeline that:

- Filters out duplicates.
- Fills null prices with 0.
- Groups by `store_id` to calculate total revenue.

Code

```
from pyspark.sql.functions import sum
final_result = (
    df
    .dropDuplicates()
    .na.fill({"price": 0})
    .groupby("store_id")
    .sum("price")
)
```

```

.groupBy("store_id")
.agg(sum("price").alias("total_revenue"))
)
final_result.show()

```

Explanation

Step	Operation	Purpose
1	dropDuplicates()	Removes duplicate records from the dataset
2	na.fill({"price": 0})	Replaces null values in the price column with 0
3	groupBy("store_id")	Groups records by store ID
4	sum("price")	Calculates total revenue for each store

Sample Output

```

+-----+-----+
|store_id|total_revenue|
+-----+-----+
|    S102|         39985|
|    S104|         12540|
|    S101|         44520|
|    S103|         39985|
+-----+-----+

```

CONCLUSION

Insights from the Assignment:

- Apache Spark provides faster data processing than traditional MapReduce through in-memory computing.
- DataFrames offer an efficient and user-friendly way to perform data cleaning and transformations.
- Removing duplicates and handling null values improves data quality and analysis accuracy.
- Aggregation functions such as sum(), avg(), min(), and max() help generate meaningful insights from large datasets.
- Grouping operations trigger shuffle processes, which are wide transformations involving data movement across partitions.